

Unit-4

Queue

QUEUE

A queue is logically a first in first out (FIFO or first come first serve) linear data structure. It is a homogeneous collection of elements in which new elements are added at one end called rear, and the existing elements are deleted from other end called front. The basic operations (Primitive Operations) also called as Queue ADT that can be performed on queue are

1. Enqueue() or Insert() (or add) an element to the queue (push) from the rear end.
2. Dequeue() or Delete() (or remove) an element from a queue (pop) from front end.
3. Front: return the object that is at the front of the queue without removing it.
4. Empty: return true if the queue is empty otherwise return false.
5. Size: returns the number of items in the queue.

3.1 TYPES OF QUEUE

- Linear Queue or Simple queue
- Circular queue
- Double ended queue (de-queue)
- Priority queue: Priority queue is generally implemented using linked list

3.2 Linear Queue or Queue

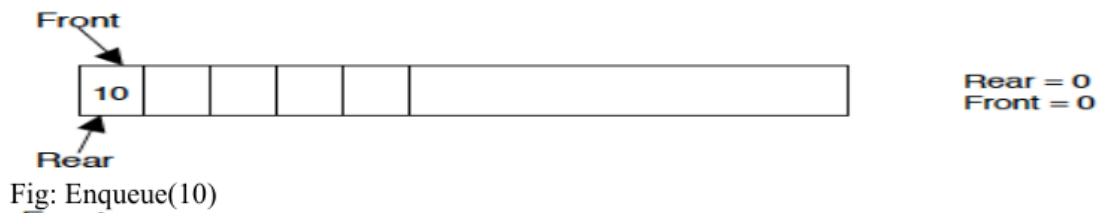
Enqueue operation will insert (or add) an element to queue, at the rear end, by incrementing the array index. Dequeue operation will delete (or remove) from the front end by decrementing the array index and will assign the deleted value to a variable. Initially front and rear is set to -1. The queue is empty whenever $\text{rear} < \text{front}$ or both the rear and front is equal to -1. Total number of elements in the queue at any time is equal to $\text{rear} - \text{front} + 1$, when implemented using arrays.

Below are the few operations in queue.

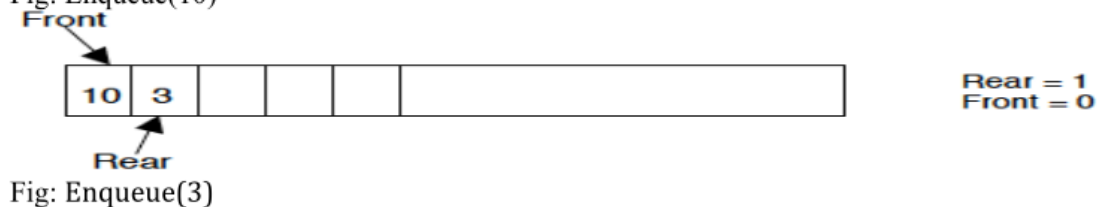


Queue is empty

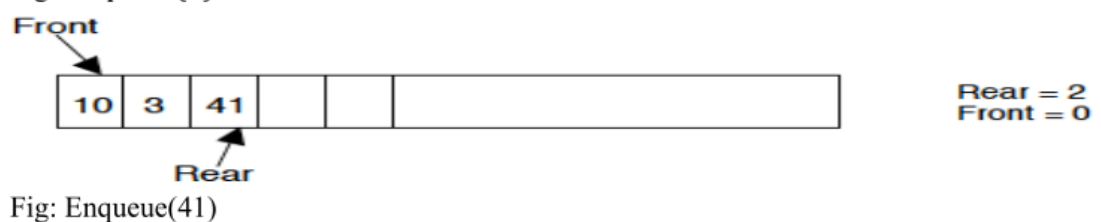
rear=-1,front=-1



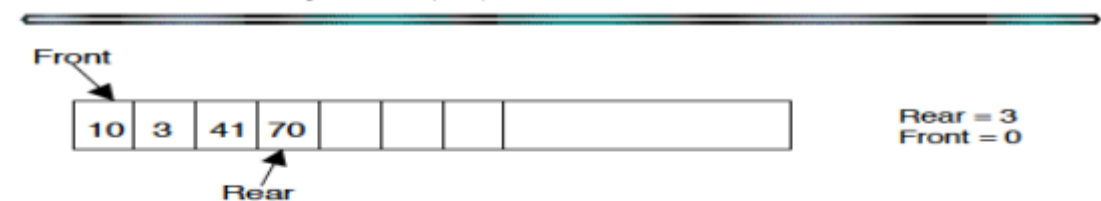
Rear = 0
Front = 0



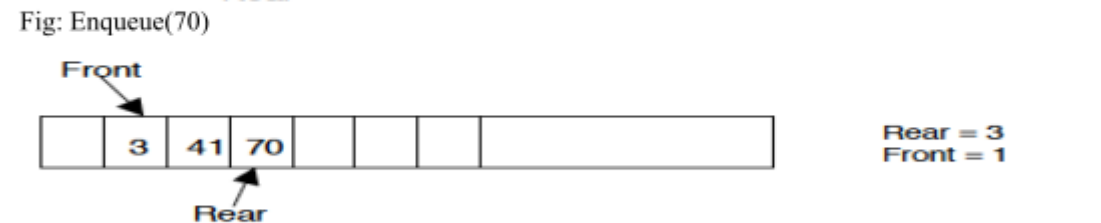
Rear = 1
Front = 0



Rear = 2
Front = 0



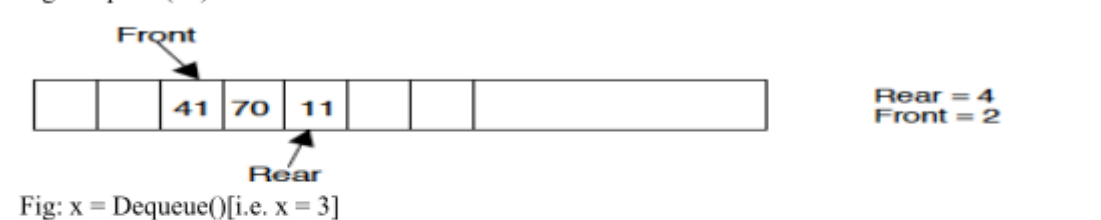
Rear = 3
Front = 0



Rear = 3
Front = 1



Rear = 4
Front = 1



Rear = 4
Front = 2



Rear = 4
Front = 3

Note. During the insertion of first element in the queue, we always increment the front by one.

Algorithm for queue operations :-

Before we implement actual operations, first follow the below steps to create an empty queue.

Step 1 - Include all the header files which are used in the program and define a constant 'SIZE' with specific value.

Step 2 - Declare all the user defined functions which are used in queue implementation.

Step 3 - Create a one dimensional array with above defined SIZE (int queue[SIZE])

Step 4 - Define two integer variables 'front' and 'rear' and initialize both with '-1'. (int front = -1, rear = -1)

Step 5 - Then implement main method by displaying menu of operations list and make suitable function calls to perform operation selected by the user on queue.

enqueue(value) - Inserting value into the queue

Step 1 - Check whether queue is FULL. (rear == SIZE-1)

Step 2 - If it is FULL, then display "Queue is FULL!!! Insertion is not possible!!!" and terminate the function.

Step 3 - If it is NOT FULL, then increment rear value by one (rear++) and set queue[rear] = value.

dequeue() - Deleting a value from the Queue

Step 1 - Check whether queue is EMPTY. (front == rear)

Step 2 - If it is EMPTY, then display "Queue is EMPTY!!! Deletion is not possible!!!" and terminate the function.

Step 3 - If it is NOT EMPTY, then increment the front value by one (front ++). Then display queue[front] as deleted element. Then check whether both front and rear are equal (front == rear), if it TRUE, then set both front and rear to '-1' (front = rear = -1).

display() - Displays the elements of a Queue

Step 1 - Check whether queue is EMPTY. (front == rear)

Step 2 - If it is EMPTY, then display "Queue is EMPTY!!!" and terminate the function.

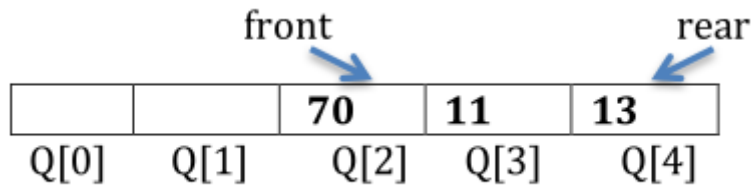
Step 3 - If it is NOT EMPTY, then define an integer variable 'i' and set 'i = front+1'.

Step 4 - Display 'queue[i]' value and increment 'i' value by one (i++). Repeat the same until 'i' value reaches to rear (i <= rear)

Array implementation of queue ==>

3.3 Circular Queue

Suppose a queue Q has maximum size 5, say 5 elements pushed and 2 elements popped.



Now if we attempt to add more elements, even though 2 queue cells are free, the elements cannot be pushed. Because in a queue, elements are always inserted at the rear end and hence rear points to last location of the queue array Q[4]. That is queue is full (overflow condition) though it is empty. This limitation can be overcome if we use circular queue. In circular queues the elements Q[0], Q[1], Q[2] Q[n - 1] is represented in a circular fashion with Q[1] following Q[n]. A circular queue is one in which the insertion of a new element is done at the very first location of the queue if the last location at the queue is full.

Suppose Q is a queue array of 6 elements. Enqueue() and Dequeue() operation can be performed on circular. The following figures will illustrate the same.

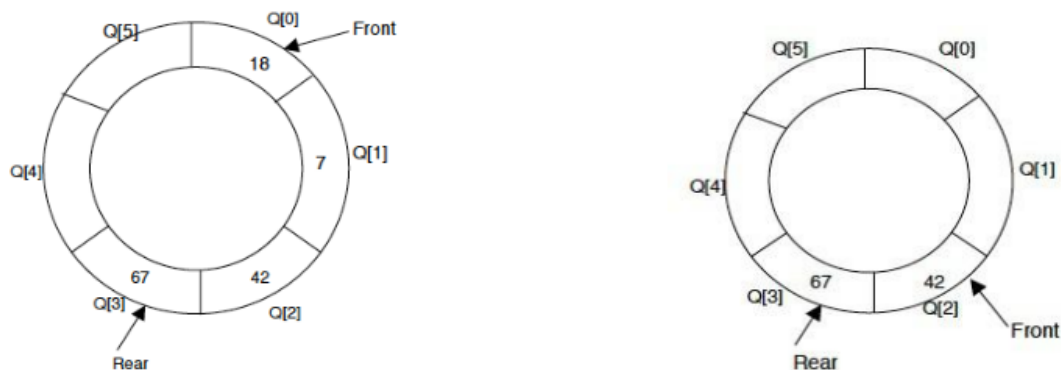


Fig1: A Circular Queue After inserting 18,7,42,67. **Fig2:** Circular Queue after popping 18,7.

After inserting an element at last location Q[5], the next element will be inserted at the very first location (i.e., Q[0]) that is circular queue is one in which the first element comes just after the last element.

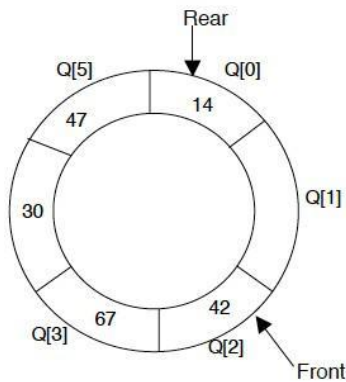


Fig3: Circular Queue after pushing 30, 47, 14

At any time the relation will calculate the position of the element to be inserted

$$\mathbf{Rear = (Rear + 1) \% SIZE}$$

After deleting an element from circular queue the position of the front end is calculated by the relation

$$\mathbf{Front = (Front + 1) \% SIZE}$$

After locating the position of the new element to be inserted, rear, compare it with front. If (rear = front), the queue has only one element.

3.3.1 Algorithms for Circular Queue

Let Q be the arrays of some specified size say SIZE. FRONT and REAR are two pointers where the

elements are deleted and inserted at two ends of the circular queue. DATA is the element to be inserted. Initialize FRONT = -1 and REAR = -1

3.3.1.1 Inserting an element to circular Queue

1.If ((FRONT is equal to 0 && rear = MAX-1) OR front = rear+1) //check for queue full

i. Display “Queue is full”

ii. Exit

2. If (FRONT is equal to - 1 && rear == -1) //Empty Queue (inserting first time)

i. FRONT = 0

ii. REAR = 0

3.Else

i. REAR = (REAR + 1) % SIZE

4.Input the value to be inserted and assign to variable “DATA”

5.Q [REAR] = DATA

6.Repeat steps 2 to 7 if we want to insert more elements

7.Exit

3.3.1.2 Deleting an element from a circular queue

1. If (FRONT is equal to -1 and rear == -1)
 - i. Display "Queue is empty"
 - ii. Exit
2. Else
 - i. DATA = Q [FRONT]
3. If (REAR is equal to FRONT) //Queue has only one element
 - i. FRONT = -1
 - ii. REAR = -1
4. Else
 - i. FRONT = (FRONT +1) % SIZE
5. Repeat the steps 1 to 4, if we want to delete more elements
6. Exit

3.3.1.3 Traversing in a Circular Queue

1. Initialize front_pos = FRONT and rear_pos = REAR
2. If front_pos is equal to -1 and rear_pos is equal to -1
 - a. Display queue empty
 - b. return
3. If front_pos is less than or equal to rear_pos [Like linear array]
 - a. Print Q[front_pos]
 - b. front_pos = front_pos+1
4. Else [eg: rear = 2 and front = 5 where MAX = 10]
 - a. Repeat until front_pos is less than or equal to MAX-1 [print upto MAX-1]
 - i. Print Q[front_pos]
 - ii. Front_pos = front_pos+1
 - b. Set front_pos = 0
 - c. Repeat until front_pos is less than or equal to rear_pos [print upto rear]
 - i. Print Q[front_pos]
5. Stop

Program to Implement Circular Queue Using Array ==> see teams

3.4 DEQUES

A deque is a homogeneous list in which elements can be added or inserted (called Enqueue operation) and deleted or removed from both the ends (which is called Dequeue operation). ie; we can add a new element at the rear or front end and also we can remove an element from both front and rear end. Hence it is called Double Ended Queue.

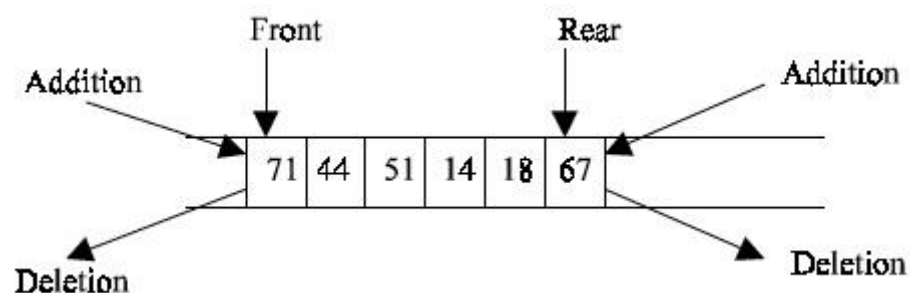
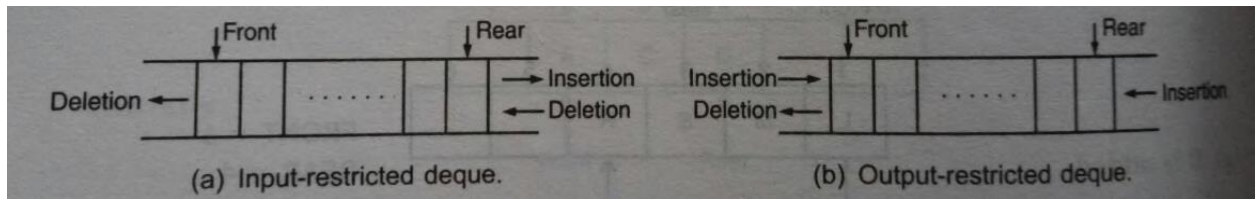


Figure 2: A Deque

There are two types of deque depending upon the restriction to perform insertion or deletion operations at the two ends. They are

1. Input restricted deque: An input restricted deque is a deque, which allows insertion at only 1 end, rear end, but allows deletion at both ends, rear and front end of the lists.

2. Output restricted deque: An output-restricted deque is a deque, which allows deletion at only one end, front end, but allows insertion at both ends, rear and front ends, of the lists.



The possible operation (Deque ADT) performed on deque is

1. Add an element at the rear end (insert_rear)
2. Add an element at the front end (insert_front)
3. Delete an element from the front end (delete_front)
4. Delete an element from the rear end (delete_rear)

Only 1st, 3rd and 4th operations are performed by input-restricted deque and 1st, 2nd and 3rd operations are performed by output-restricted deque.

3.4.1 Algorithms for Inserting an Element in Deque

Let Q be the array of MAX elements. front (or left) and rear (or right) are two array index (pointers), where the addition and deletion of elements occurred. Let DATA be the element to be inserted. Before inserting any element to the queue left and right pointer will point to the -1.

3.4.1.1 Insert an element at the right side (rear end) of the de-queue

1. Input the DATA to be inserted
2. If $((\text{front} == 0 \ \&\& \ \text{rear} == \text{MAX}-1) \ || \ (\text{front} == \text{rear} + 1))$
 - i. Display "Queue Overflow"
 - ii. Exit
3. If $(\text{front} == -1 \ \&\& \ \text{rear} == -1)$
 - i. $\text{front} = 0$
 - ii. $\text{rear} = 0$
4. Else
 - i. if $(\text{rear} != \text{MAX} - 1)$ //Deque already contain more than one element
 - a) $\text{rear} = \text{rear} + 1$
 - ii. Else //rear has reach at the end of array

[Shift all the elements from position front until rear back by 1 position]

for(i = front; i<=rear;i++)

Q[i-1] = Q[i]

5.Q [rear] = DATA

6.Exit

3.4.1.2 Insert An Element At The Left Side (Front) Of The De-Queue

1. Input the DATA to be inserted
2. If ((front == 0 && rear == MAX-1) || (front == rear+1))
 - i. Display "Queue Overflow"
 - ii. Exit
3. If (front == -1 && rear == -1)
 - i. front = 0
 - ii. rear = 0
 - iii. Q [front] = DATA
4. Else
 - i. if (front != 0) // Deque already contain more than one element
 - a) front = front- 1
 - b) Q [front] = DATA
 - ii. Else
 - a. Shift all the elements from position front until rear ahead by 1 position
for(i=rear; i>=front;i--)
Q[i+1] = Q [i]
 - b. Q[front] = DATA
5. Exit

3.4.2 Algorithms for deleting an element from Deque

Let Q be the array of MAX elements. front (or left) and rear (or right) are two array index (pointers), where the addition and deletion of elements occurred. DATA will contain the element just deleted.

3.4.2.1 Delete An Element From The Right Side (Rear side) Of The De-Queue

1. If (front == -1 && rear == -1)
 - i. Display "Queue Underflow"
 - ii. Exit
2. If (front == rear) //Deque has only one element (last element)
 - i) DATA = Q [rear]
 - a) front = - 1
 - b) rear = - 1
3. Else //deque has more than one element
 - i. DATA = Q [rear]
 - a) rear = rear -1
4. Exit

3.4.2.2 Delete An Element From The Left Side (Front Side) Of The De-Queue

1.If (front == -1 && rear == -1)

i. Display "Queue Underflow"

ii. Exit

2. If(front == rear)

i) $DATA = Q[front]$

a) $front = - 1$

b) $rear = - 1$

3. Else // Dequeue has more than one element

i) $DATA = Q[front]$

a) $Front = front + 1$

4. Exit

3.5 PRIORITY QUEUES

Priority Queue is a queue where each element is assigned a priority. The priority queue is a special queue and an abstract data type, this is a type of queue in which each element is associated with a priority and the elements are arranged in the queue as per their priority.

Properties of priority queue:

A priority queue is an extended version of the normal queue with the only difference being that its elements have a priority value associated with them. So the priority queue has a few properties, which are as follows:

- Every element of the queue has a priority assigned to it.
- The element with the highest priority is served first or will be dequeued before the element having low priority.
- Two elements with the same priority appear, then they will be served on the basis of their occurrence. The one who occurred first will be dequeued first.

Types of Priority Queue

Priority Queue is of two types:

- Ascending order Priority

In Ascending order priority queue, the element with least value has the highest priority.

- Descending order Priority

In Descending order priority queue, the element with the highest value has the highest priority.

For example, Consider a manager who is in a process of checking and approving files in a first come first serve basis. In between, if any urgent file (with a high priority) comes, he will process the urgent file next and continue with the other low urgent files.

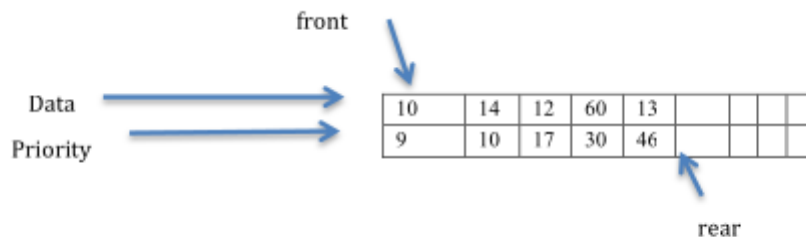
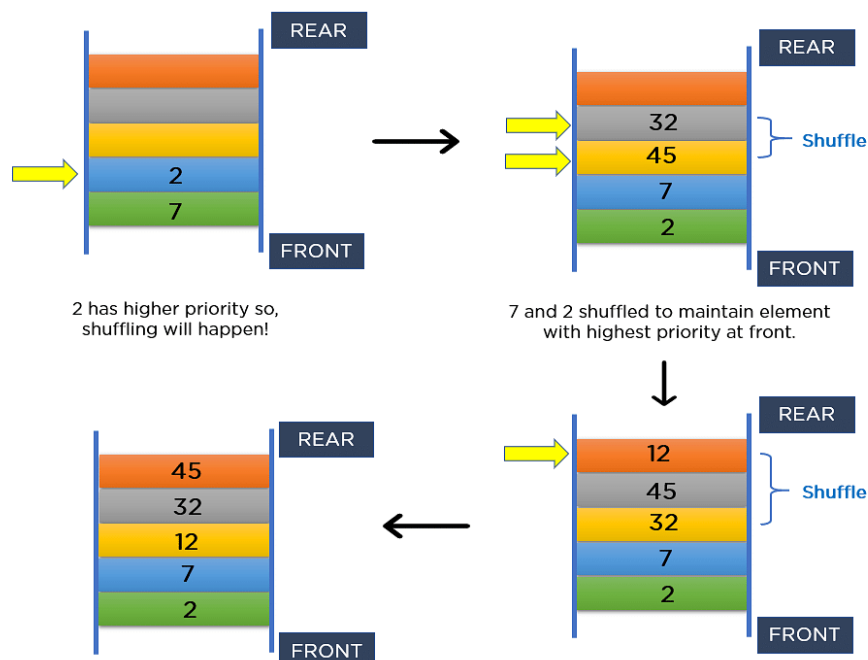


Figure 4: Priority Queue representation using Array

Above Figure 3 gives a pictorial representation of priority queue using arrays after adding 5 elements (10,14,12,60,13) with its corresponding priorities (9,10,17,30,46). Here the priorities of the data(s) are in ascending order. Always we may not be pushing the data in an ascending order. From the mixed priority list it is difficult to find the highest priority element if the priority queue is implemented using arrays. Moreover, the implementation of priority queue using array will yield n comparisons (in liner search), so the time complexity is much higher than the other queue for inserting an element. So it is always better to implement the priority queue using linked list - where a node can be inserted at anywhere in the list.



3.6 Application of Queues

- Round robin techniques for processor, scheduling is implemented using queue.
- Printer server routines (in drivers) are designed using queues.

- All types of customer service software (like Railway/Air ticket reservation) are designed using queue to give proper service to the customers.
- Disk Driver: maintains a queue of disk input/output requests
- Scheduler (e.g, in operating system): maintains a queue of processes awaiting a slice of machine time.
- Call center phone systems will use a queue to hold people in line until a service representative is free.
- Buffers on MP3 players and portable CD players, iPod playlist. Playlist for jukebox - add songs to the end, play from the front of the list.